AI-BASED PHISHING EMAIL DETECTION SYSTEM

MINI PROJECT SYNOPSIS

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering – AIML

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by

Student Name: ***Gunja Kumari***

Roll No.: ***2401531530050***

Section: ***3F***

Under the Supervision of

Guide Name: ***Mr. Yogendra***

Assistant Professor

Department of Computer Science & Engineering

Lloyd Institute of Engineering & Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

INDEX

Sr. No.	*Topic*	*Page No.*
01	Introduction	3 – 4
02	Statement of Problem with Objective	5
03	Literature Survey	6
04	Expected Outcome / Scope of the Project	7
05	Tentative Work Plan (mapped to the 10-week timeline)	8
06	Software/Hardware required for the development of the project	9
07	References	10

_____ _____

Student's Signature ***Guide's Signature***

1. INTRODUCTION

Electronic mail remains one of the most widely used communication channels in both personal and professional environments. Unfortunately, it is also one of the most exploited vectors for cyber-attacks. Phishing is a form of social engineering in which an attacker masquerades as a trustworthy entity and sends fraudulent emails designed to steal sensitive information such as passwords, banking credentials, credit-card numbers, or personal identification data. According to industry threat reports, phishing continues to account for the majority of reported data breaches worldwide, causing enormous financial and reputational damage to individuals and organisations alike.

The proposed project, ***AI-Based Phishing Email Detection System***, is a full-stack web application developed in the domain of Cyber Security and Natural Language Processing (NLP). The application analyses the textual content of an email and classifies it as either legitimate or phishing. Unlike conventional detection tools that rely on a single technique, this system employs a ***Hybrid Detection Engine*** that combines a Machine Learning text-classification model with a Rule-Based Heuristic system. The machine learning component provides a data-driven probability score, while the heuristic component scans the text for known phishing indicators such as urgency words, suspicious Uniform Resource Locators (URLs), and requests for account credentials. Every heuristic flag artificially boosts the phishing probability, ensuring that obvious phishing attempts are caught even when the statistical model is uncertain.

A distinguishing feature of the system is its support for ***explainable AI (XAI)***. Instead of merely returning a binary "Phishing" or "Safe" label, the system presents the user with a visual confidence score and a bulleted list of the specific rule-based indicators that influenced the decision, allowing the user to understand exactly why an email was flagged.

The project also integrates ***Optical Character Recognition (OCR)*** technology. Users may paste raw email text directly into the application, or alternatively upload or paste a screenshot of an email (for example, a screen capture received over a messaging application). The frontend performs client-side image preprocessing: it computes the average brightness of the image, and if a dark-mode screenshot is detected, the colours are automatically inverted and the contrast is boosted before OCR is applied, which considerably improves text-extraction accuracy. This multi-modal input capability makes the tool practical for real-world scenarios where the original email text is not directly available.

1.1 Technology Used

The frontend is built with ***React.js*** bundled by ***Vite***, styled using ***Tailwind CSS***, and uses ***Tesseract.js*** for client-side OCR. The backend is developed in ***Python*** using the ***Flask*** micro-framework, which exposes a RESTful ***Application Programming Interface (API)*** through a /api/predict POST endpoint. The machine learning pipeline is implemented with ***Scikit-Learn***: textual features are extracted using ***Term Frequency–Inverse Document Frequency (TF-IDF)*** vectorization, considering unigrams and bigrams, and classification is performed using ***Logistic Regression***, which is well suited to text classification and probability estimation. The trained pipeline is serialized to disk using ***Joblib*** and loaded at inference time.

1.2 Important Technical Terms

• ***Phishing:*** A fraudulent attempt to obtain sensitive information by disguising as a trustworthy entity in an electronic communication.

• ***TF-IDF:*** A statistical measure that evaluates how relevant a word is to a document within a corpus, by weighting term frequency against inverse document frequency.

• ***Unigram / Bigram:*** A single word, or a contiguous pair of words, used as a feature for text representation.

• ***Logistic Regression:*** A supervised machine learning algorithm that estimates the probability of a binary outcome using a logistic function.

• ***OCR:*** Optical Character Recognition – the conversion of images containing text into machine-readable text.

• ***Heuristic:*** A rule of thumb, in this context a regular-expression or keyword-based rule, used to detect known phishing patterns.

• ***Explainable AI (XAI):*** Methods that make the decisions of AI systems understandable to humans by revealing the reasoning behind a prediction.

• ***RESTful API:*** An architectural style for web services in which clients exchange data with a server using standard HTTP methods such as POST.

• ***CORS:*** Cross-Origin Resource Sharing – a mechanism that allows a web page to request resources from a different domain.

• ***Glassmorphism:*** A modern UI design style based on translucent, blurred surfaces; used here in the dark-mode frontend.

2. STATEMENT OF PROBLEM AND OBJECTIVES

2.1 Statement of Problem

Existing email security solutions suffer from several limitations. Purely rule-based spam filters (e.g., keyword blacklists and sender blacklists) are easily evaded by attackers through obfuscation, misspellings, and newly registered domains. Machine learning–based classifiers, on the other hand, frequently act as opaque "**black boxes**": they return a verdict without any justification, so users cannot verify whether the classification is trustworthy. Furthermore, most available detection tools accept only raw email text and cannot analyse emails that are presented in the form of screenshots or forwarded images, which is an extremely common way in which suspicious emails are shared among users. Finally, many commercial solutions are expensive and closed-source, offering no transparency or scope for customisation. There is therefore a clear need for an affordable, transparent, multi-modal, and explainable phishing detection system.

2.2 Objectives of the Proposed Project

1. To develop a web-based application capable of classifying email text as phishing or legitimate with high accuracy.

2. To design a hybrid detection engine that combines a TF-IDF + Logistic Regression machine learning model with a rule-based heuristic scoring system, where each triggered heuristic boosts the phishing probability by 15% (capped at 99%).

3. To integrate Optical Character Recognition (Tesseract.js) with intelligent preprocessing so that phishing emails supplied as screenshots – including dark-mode captures – can be analysed.

4. To provide an explainable AI output consisting of a visual confidence score and a list of the specific rule-based indicators that triggered the warning.

5. To deliver a modern, responsive user interface using React and Tailwind CSS with real-time feedback and micro-animations.

6. To package the system as a lightweight, open, and customisable alternative to costly commercial email security products.

3. LITERATURE SURVEY

A considerable body of research exists on the automated detection of phishing emails. The work described below has directly influenced the design of the proposed system.

- ***Khonji, Iraqi and Jones (2013):*** Presented a comprehensive survey of phishing detection techniques and classified them into content-based, blacklist-based, and machine learning–based approaches, concluding that hybrid and content-aware methods are the most promising direction – a conclusion adopted in the present project.
- ***Toolan and Carthy (2010):*** Demonstrated that classifier ensembles combining multiple machine learning algorithms outperform individual classifiers for phishing email detection, motivating the combination of statistical and rule-based techniques.
- ***Verma and Dasgupta (2015):*** Reviewed machine learning and feature-based approaches for tackling phishing emails, highlighting TF-IDF feature extraction as an effective representation of email content for classification.
- ***Altwaijry et al. (2022):*** Surveyed phishing detection using classical machine learning, confirming that algorithms such as Logistic Regression and Naive Bayes, trained on TF-IDF features, achieve high accuracy on large labelled phishing email datasets.
- ***SpamAssassin (open-source project):*** The widely deployed Apache SpamAssassin filter applies hundreds of hand-crafted heuristic rules to score email content. The rule-based indicator subsystem of the proposed project draws inspiration from this proven approach, while extending it with an ML model and explainable output.

Review of the above work reveals a persistent gap: existing systems either provide accurate but unexplainable predictions, or explainable but easily evaded static rules, and very few accept non-textual input such as screenshots. The proposed system addresses this gap by fusing the two paradigms in a single hybrid engine and adding multi-modal (text + image) input with OCR.

4. EXPECTED OUTCOME / SCOPE OF THE PROJECT

4.1 Expected Outcomes

• A fully functional full-stack web application that accepts email text or email screenshots and returns an accurate phishing/legitimate verdict within seconds.

• A trained and serialised Scikit-Learn pipeline (***phishing-mail-v1.joblib***) achieving high classification accuracy on a large labelled dataset of legitimate and phishing emails.

• A hybrid confidence score combining ML probability with heuristic boosting, with an intuitive progress-bar visualisation.

• An explainable report listing every triggered indicator (urgency/threat words, suspicious URLs, credential requests, reward claims) for each analysed email.

• Reliable OCR extraction from both light-mode and dark-mode screenshots owing to the automatic brightness analysis, inversion, and contrast-boost preprocessing.

4.2 Key Features and Intended Users

Key features include multi-modal input, real-time analysis, explainable AI output, heuristic boosting, and a premium dark-mode glassmorphism interface. The intended users are individual internet users, students, small and medium enterprises without dedicated security infrastructure, and IT help-desks that need a quick, transparent tool for triaging suspicious emails.

4.3 Practical Applications and Future Scope

In the short term the system can serve as a browser-accessible utility for verifying suspicious emails. Future extensions include: analysing email headers and sender-domain reputation; browser extension and email-client plugin integration for automatic scanning of inboxes; multilingual phishing detection; deployment of deep-learning models such as LSTM or transformer-based classifiers; URL reputation checking through external threat-intelligence APIs; and a feedback mechanism that lets users report missed phishing attempts, enabling continuous retraining and improvement of the model.

5. TENTATIVE WORK PLAN

The project is planned to be completed over a period of ten weeks as per the following schedule:

Week	***Major Activities and Milestones***
Week 1	Requirement analysis; finalisation of project scope, features, and technology stack; synopsis preparation.
Week 2	Literature study of phishing detection techniques and machine learning approaches; dataset collection and review.
Week 3	Data preprocessing; implementation of the TF-IDF vectorization and Logistic Regression pipeline in train.py; initial model training.
Week 4	Model evaluation and tuning; serialisation of the trained pipeline with Joblib.
Week 5	Backend development: Flask server, /api/predict REST endpoint, heuristic indicator extraction module, and CORS configuration.
Week 6	Frontend development: React + Tailwind CSS user interface with text-input and image-upload flows.
Week 7	OCR integration: Tesseract.js setup, image preprocessing (grayscale, dark-mode inversion, contrast boost), and end-to-end API connection.
Week 8	Result rendering: confidence score visualisation, indicator list display, and UI polish with micro-animations.
Week 9	Testing and debugging: unit testing, cross-browser testing, accuracy benchmarking, and performance optimisation.
Week 10	Documentation, report writing, and final presentation preparation with demonstration.

6. SOFTWARE / HARDWARE REQUIRED

6.1 Software Requirements

Component	***Details***
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS
Programming Languages	Python 3.x (backend, ML), JavaScript / JSX (frontend)
Frontend Framework	React.js with Vite, Tailwind CSS v4, Lucide-React icons
Backend Framework	Flask (WSGI), Flask-CORS
Machine Learning Libraries	Scikit-Learn, Pandas, NumPy, Joblib
OCR Engine	Tesseract.js (client-side)
IDE / Tools	Visual Studio Code / PyCharm, Git, Node.js and npm
Dataset	Labelled legitimate/phishing email dataset (dataset.csv)
Browser	Any modern browser (Chrome / Edge / Firefox) supporting ES6 and Canvas API

6.2 Hardware Requirements

Component	***Minimum Specification***
Processor	Intel Core i5 / AMD Ryzen 5 or higher (2.10 GHz+)
RAM	Minimum 8 GB (16GB recommended)
Storage	Minimum 10 GB of free disk space
Display	Standard display with 1366 × 768 resolution or higher
Other	Stable internet connection for package installation and dataset download

7. REFERENCES

- Altwaijry, I., Al-Turaiki, I., Alrashed, S., and others, "**Phishing Detection Using Machine Learning: A Literature Review,**" IEEE Access, vol. 10, pp. 135–143, 2022.

- Apache Software Foundation, "Apache SpamAssassin Project Documentation," 2026. [Online]. Available: **https://spamassassin.apache.org/**

- Hastie, T., Tibshirani, R., and Friedman, J., **The Elements of Statistical Learning: Data Mining,** Inference, and Prediction, 2nd ed., New York: Springer, 2009.

- Khonji, M., Iraqi, Y., and Jones, A., "**Phishing Detection: A Literature Survey**," IEEE Communications Surveys & Tutorials, vol. 15, no. 4, pp. 2091–2121, 2013.

- Pedregosa, F., and others, "**Scikit-Learn: Machine Learning in Python**," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

- Smith, R. J., "**An Overview of the Tesseract OCR Engine**," in Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), pp. 629–633, 2007.

- Toolan, F., and Carthy, J., "**Phishing Detection Using Classifier Ensembles**," in Proceedings of the International Conference on Email and Anti-Spam (CEAS), 2010.

- Verma, R., and Dasgupta, D., "**Tackling Phishing Emails Using Machine Learning and Data Sensitivity**," in Proceedings of the 2015 ACM International Workshop on Security and Privacy Analytics, pp. 45–53, 2015.

- Mozilla Developer Network, "**Canvas API**," MDN Web Docs, 2026. [Online]. Available: **https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API**

- Pallets Projects, "**Flask Documentation**," 2026. [Online]. Available: **https://flask.palletsprojects.com/**

- ***React, "React Documentation," 2026. [Online]. Available: https://react.dev/***